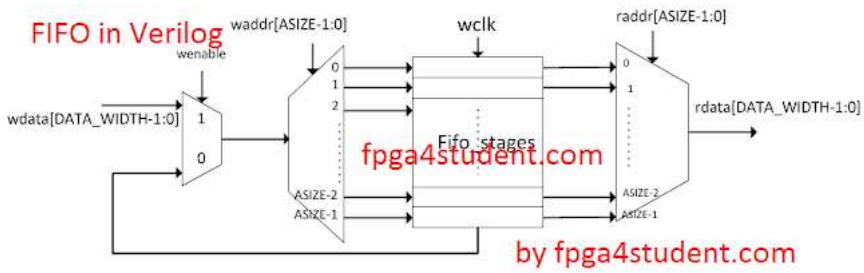
Verilog code for FIFO memory

In this project, Verilog code for FIFO memory is presented. The First-In-First-Out (FIFO) memory with the following specification is implemented in Verilog:

- **16 stages**
- **8-bit data width**
- **Status signals:**
  - **Full: high when FIFO is full else low.**
  - **Empty: high when FIFO is empty else low.**
  - **Overflow: high when FIFO is full and still writing data into FIFO, else low.**
  - **Underflow: high when FIFO is empty and still reading data from FIFO, else low.**
  - **Threshold: high when the number of data in FIFO is less than a specific threshold, else low.**



FIFO architecture

Verilog code for the FIFO memory:

```
// fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for FIFO memory
// Top level Verilog code for FIFO Memory
module fifo_mem(data_out,fifo_full, fifo_empty, fifo_threshold, fifo_overflow, fif
  input wr, rd, clk, rst_n;
  input[7:0] data_in;   // FPGA projects using Verilog/ VHDL
  output[7:0] data_out;
  output fifo_full, fifo_empty, fifo_threshold, fifo_overflow, fifo_underflow;
  wire[4:0] wptr,rptr;
  wire fifo_we,fifo_rd;
  write_pointer top1(wptr,fifo_we,wr,fifo_full,clk,rst_n);
  read_pointer top2(rptr,fifo_rd,rd,fifo_empty,clk,rst_n);
  memory_array top3(data_out, data_in, clk,fifo_we, wptr,rptr);
  status_signal top4(fifo_full, fifo_empty, fifo_threshold, fifo_overflow, fifo_un
 endmodule
// fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for FIFO memory
// Verilog code for Memory Array submodule
 module memory_array(data_out, data_in, clk,fifo_we, wptr,rptr);
  input[7:0] data_in;
  input clk,fifo_we;
  input[4:0] wptr,rptr;
  output[7:0] data_out;
  reg[7:0] data_out2[15:0];
  wire[7:0] data_out;
  always @(posedge clk)
  begin
   if(fifo_we)
      data_out2[wptr[3:0]] <=data_in ;
  end
  assign data_out = data_out2[rptr[3:0]];
```

```
 endmodule
// fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for FIFO memory
// Verilog code for Read Pointer sub-module
 module read_pointer(rptr,fifo_rd,rd,fifo_empty,clk,rst_n);
  input rd,fifo_empty,clk,rst_n;
  output[4:0] rptr;
  output fifo_rd;
  reg[4:0] rptr;
  assign fifo_rd = (~fifo_empty)& rd;
  always @(posedge clk or negedge rst_n)
  begin
   if(~rst_n) rptr <= 5'b000000;
   else if(fifo_rd)
    rptr <= rptr + 5'b000001;
   else
    rptr <= rptr;
  end
 endmodule
// fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for FIFO memory
// Verilog code for Status Signals sub-module
 module status_signal(fifo_full, fifo_empty, fifo_threshold, fifo_overflow, fifo_u
  input wr, rd, fifo_we, fifo_rd,clk,rst_n;
  input[4:0] wptr, rptr;
  output fifo_full, fifo_empty, fifo_threshold, fifo_overflow, fifo_underflow;
  wire fbit_comp, overflow_set, underflow_set;
  wire pointer_equal;
  wire[4:0] pointer_result;
  reg fifo_full, fifo_empty, fifo_threshold, fifo_overflow, fifo_underflow;
  assign fbit_comp = wptr[4] ^ rptr[4];
  assign pointer_equal = (wptr[3:0] - rptr[3:0]) ? 0:1;
  assign pointer_result = wptr[4:0] - rptr[4:0];
  assign overflow_set = fifo_full & wr;
  assign underflow_set = fifo_empty&rd;
  always @(*)
  begin
   fifo_full =fbit_comp & pointer_equal;
   fifo_empty = (~fbit_comp) & pointer_equal;
   fifo_threshold = (pointer_result[4]||pointer_result[3]) ? 1:0;
  end
  always @(posedge clk or negedge rst_n)
  begin
  if(~rst_n) fifo_overflow <=0;
  else if((overflow_set==1)&&(fifo_rd==0))
   fifo_overflow <=1;
   else if(fifo_rd)
    fifo_overflow <=0;
    else
     fifo_overflow <= fifo_overflow;
  end
  always @(posedge clk or negedge rst_n)
  begin
  if(~rst_n) fifo_underflow <=0;
  else if((underflow_set==1)&&(fifo_we==0))
   fifo_underflow <=1;
   else if(fifo_we)
    fifo_underflow <=0;
    else
     fifo_underflow <= fifo_underflow;
  end
 endmodule
// fpga4student.com: FPga projects, Verilog projects, VHDL projects
```

```
// Verilog project: Verilog code for FIFO memory
// Verilog code for Write Pointer sub-module
 module write_pointer(wptr,fifo_we,wr,fifo_full,clk,rst_n);
  input wr,fifo_full,clk,rst_n;
  output[4:0] wptr;
  output fifo_we;
  reg[4:0] wptr;
  assign fifo_we = (~fifo_full)&wr;
  always @(posedge clk or negedge rst_n)
  begin
   if(~rst_n) wptr <= 5'b000000;
   else if(fifo_we)
    wptr <= wptr + 5'b000001;
   else
    wptr <= wptr;
  end
 endmodule
```

Verilog testbench for the FIFO memory:

```
// 1. The timescale directive
 `timescale     10 ps/ 10 ps
// fpga4student.com: FPga projects, Verilog projects, VHDL projects
// Verilog project: Verilog code for FIFO memory
// Verilog Testbench code for FIFO memory
 // 2. Preprocessor Directives
 `define          DELAY 10
 // 3. Include Statements
 //`include     "counter_define.h"
 module     tb_fifo_32;
 // 4. Parameter definitions
 parameter     ENDTIME      = 40000;
 // 5. DUT Input regs
 reg     clk;
 reg     rst_n;
 reg     wr;
 reg     rd;
 reg     [7:0] data_in;
 // 6. DUT Output wires
 wire     [7:0] data_out;
 wire     fifo_empty;
 wire     fifo_full;
 wire     fifo_threshold;
 wire     fifo_overflow;
 wire     fifo_underflow;
 integer i;
 // 7. DUT Instantiation
// fpga4student.com: FPga projects, Verilog projects, VHDL projects
 fifo_mem tb (/*AUTOARG*/
   // Outputs
   data_out, fifo_full, fifo_empty, fifo_threshold, fifo_overflow,
   fifo_underflow,
   // Inputs
   clk, rst_n, wr, rd, data_in
   );
 // 8. Initial Conditions
 initial
      begin
           clk     = 1'b0;
           rst_n     = 1'b0;
           wr     = 1'b0;
           rd     = 1'b0;
           data_in     = 8'd0;
      end
```

```
// 9. Generating Test Vectors
initial
     begin
          main;
     end
task main;
     fork
          clock_generator;
          reset_generator;
          operation_process;
          debug_fifo;
          endsimulation;
     join
endtask
task clock_generator;
     begin
          forever #`DELAY clk = !clk;
     end
endtask
task reset_generator;
     begin
          #(`DELAY*2)
          rst_n = 1'b1;
          # 7.9
          rst_n = 1'b0;
          # 7.09
          rst_n = 1'b1;
     end
endtask
task operation_process;
     begin
          for (i = 0; i < 17; i = i + 1) begin: WRE
               #(`DELAY*5)
               wr = 1'b1;
               data_in = data_in + 8'd1;
               #(`DELAY*2)
               wr = 1'b0;
          end
          #(`DELAY)
          for (i = 0; i < 17; i = i + 1) begin: RDE
               #(`DELAY*2)
               rd = 1'b1;
               #(`DELAY*2)
               rd = 1'b0;
          end
     end
endtask
// 10. Debug fifo
task debug_fifo;
     begin
          $display("-----------------------------------------------");
          $display("------------------   -----------------------");
          $display("----------- SIMULATION RESULT ----------------");
          $display("--------------       -------------------");
          $display("----------------     ---------------------");
          $display("-----------------------------------------------");
          $monitor("TIME = %d, wr = %b, rd = %b, data_in = %h",$time, wr, rd, dat
     end
endtask
// 11. Self-Checking
reg [5:0] waddr, raddr;
reg [7:0] mem[64:0];
always @ (posedge clk) begin
```

```
        if (~rst_n) begin
            waddr     <= 6'd0;
        end
        else if (wr) begin
            mem[waddr] <= data_in;
            waddr <= waddr + 1;
        end
        $display("TIME = %d, data_out = %d, mem = %d",$time, data_out,mem[raddr]);
        if (~rst_n) raddr     <= 6'd0;
        else if (rd & (~fifo_empty)) raddr <= raddr + 1;
        if (rd & (~fifo_empty)) begin
            if (mem[raddr]
             == data_out) begin
                $display("=== PASS ===== PASS ==== PASS ==== PASS ===");
                if (raddr == 16) $finish;
            end
            else begin
                $display ("=== FAIL ==== FAIL ==== FAIL ==== FAIL ===");
                $display("-------------- THE SIMUALTION FINISHED ------------");
                $finish;
            end
        end
    end
    //12. Determines the simulation limit
    task endsimulation;
        begin
            #ENDTIME
            $display("-------------- THE SIMUALTION FINISHED ------------");
            $finish;
        end
    endtask
    endmodule
```

The Verilog testbench is to debug and verify if the FIFO correctly operates. Besides, it is necessary to take a look at the simulation waveform and memory to see how data flows.

VHDL code for the FIFO Memory here

Recommended Verilog projects:

1. **What is an FPGA? How Verilog works on FPGA**
2. Verilog code for FIFO memory
3. Verilog code for 16-bit single-cycle MIPS processor
4. Programmable Digital Delay Timer in Verilog HDL
5. Verilog code for basic logic components in digital circuits
6. Verilog code for 32-bit Unsigned Divider
7. Verilog code for Fixed-Point Matrix Multiplication
8. Plate License Recognition in Verilog HDL
9. Verilog code for Carry-Look-Ahead Multiplier
10. Verilog code for a Microcontroller
11. Verilog code for 4x4 Multiplier
12. Verilog code for Car Parking System
13. Image processing on FPGA using Verilog HDL
14. How to load a text file into FPGA using Verilog HDL
15. Verilog code for Traffic Light Controller
16. Verilog code for Alarm Clock on FPGA
17. Verilog code for comparator design
18. Verilog code for D Flip Flop
19. Verilog code for Full Adder
20. Verilog code for counter with testbench
21. Verilog code for 16-bit RISC Processor
22. Verilog code for button debouncing on FPGA
23. How to write Verilog Testbench for bidirectional/ inout ports

24. Tic Tac Toe Game in Verilog and LogiSim
25. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-1)
26. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-2)
27. 32-bit 5-stage Pipelined MIPS Processor in Verilog (Part-3)
28. Verilog code for Decoder
29. Verilog code for Multiplexers
30. N-bit Adder Design in Verilog
31. Verilog vs VHDL: Explain by Examples
32. Verilog code for Clock divider on FPGA
33. How to generate a clock enable signal in Verilog
34. Verilog code for PWM Generator
35. Verilog coding vs Software Programming
36. Verilog code for Moore FSM Sequence Detector

37. Verilog code for 7-segment display controller on Basys 3 FPGA

DMCA PROTECTED

Privacy Policy | Disclaimer | Sitemap | Contact | Advertise Here
Copyright © 2016-2020 FPGA4student.com All Rights Reserved.